Lab(Option B: JavaScript): Creating a Swagger Documentation for REST API

Skills Network

Estimated Time: 45 minutes

In this lab, you will understand how to create a Swagger documentation for your REST APIs using Node.js and Express.

Learning objectives

After completing this lab, you will be able to:

- Use the Swagger Editor to create Swagger documentation for REST API
- Use SwaggerUI to access the REST API endpoints of an application
- Generate code with the Swagger documentation

Prerequisites

- You must be familiar with Docker applications and commands
- You must have a good understanding of REST API.
- Knowledge of JavaScript and Node.js is highly recommended

Task 1 - Getting your application started

1. Open a terminal window by using the top menu in the IDE: **Terminal** > **New Terminal**, if you don't have one open already.
2. In the terminal, clone the repository that has the Swagger documentation and the REST API code ready by pasting the following command. The repository that you clone has code that will run a REST API application, which can be used to organize tasks.

```
git clone https://github.com/ibm-developer-skills-network/ihsgq-crud_microservices_js.git
```

3. Change the working directory to **ihsgq-crud_microservices_js/swagger_example** by running the following command.

```
cd ihsgq-crud_microservices_js/swagger_example
```

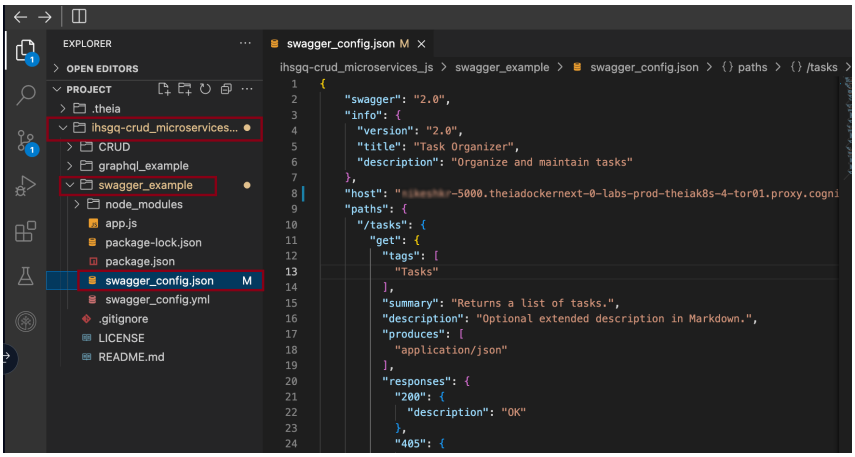
4. Run the following commands to install the required packages.

```
npm install express cors swagger-ui-express
```

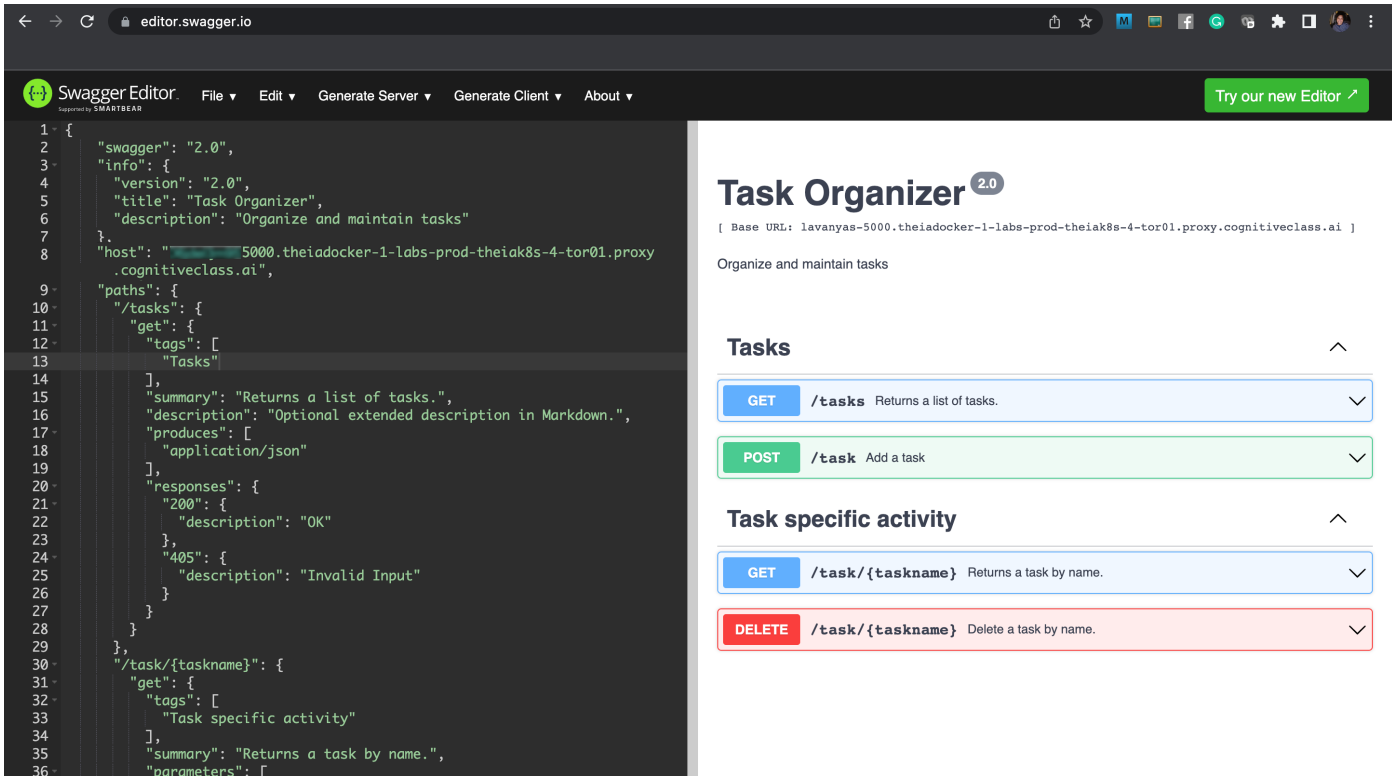
5. Now start the application which serves the REST API on port number 5000.

```
node app.js
```

6. Click on the Skills Network button on the left, it will open the "Skills Network Toolbox." Then, click Launch Application; from there, you enter the port no. as 5000 and click Your Application button. This will open a new browser page, which accesses the application you just ran.
7. Copy the URL on the address bar.
8. From the file menu, go to **ihsgq-crud_microservices_js/swagger_example/swagger_config.json** to view the file on the file editor.

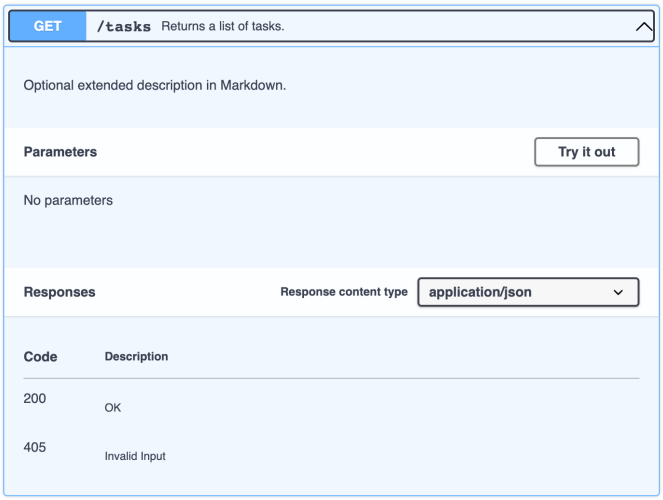


9. In the file editor, paste the application URL that you copied where it says `**<Your application URL>**` without the protocol (https://) and do not put a "/" at the end of the URL and save the file.
10. Copy the entire content of the file **swagger_config.json**. You will need this copied content to generate SwaggerUI.
11. Click on this link https://editor.swagger.io/ to go to the Swagger Editor.
12. From the File menu, click on Clear Editor to clear the content of the Swagger Editor.
13. Paste the content you copied from **swagger_config.json** on the left side. You will get a prompt that says `Would you like to convert your JSON into YAML?`. Press `Cancel` to paste the content.
14. You will see that the UI is automatically populated on the right.

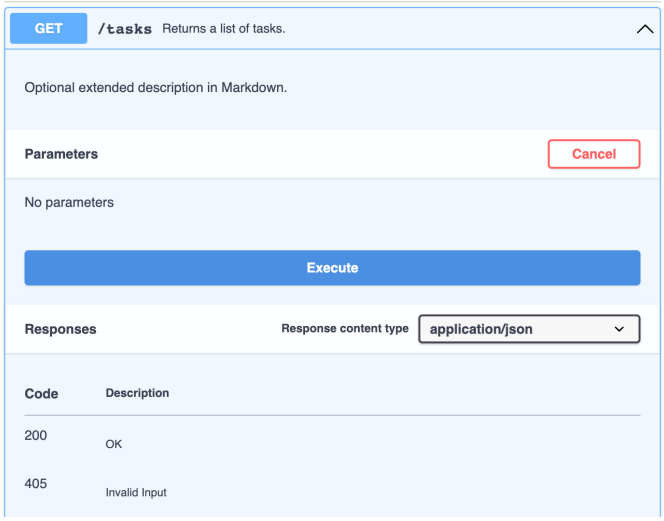


15. Now you can test each of the endpoints. Four tasks have been already added for you, when the application was started. Click on the down arrow next to **GET /tasks**.

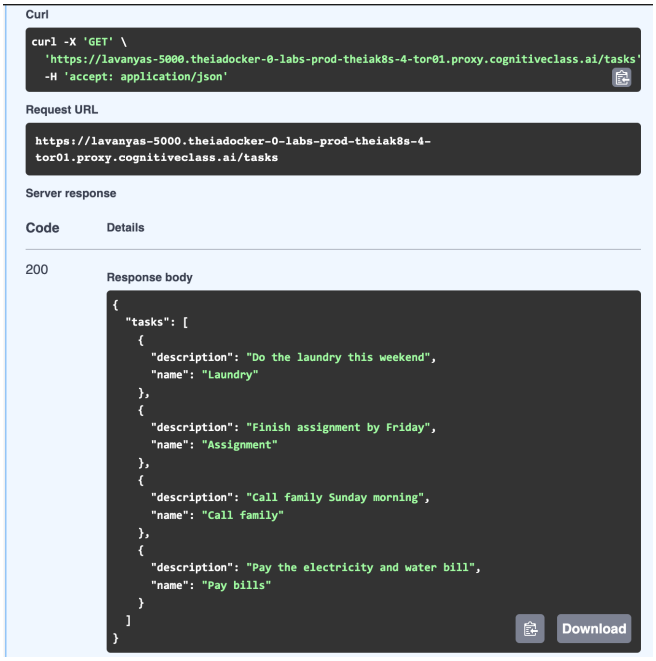
16. Click **Try it out**. This will allow you to try your REST API endpoint.



17. Click **Execute** to invoke a call to your REST API. This is a **GET** request that does not take any parameters. It returns the task as an **application/json**.



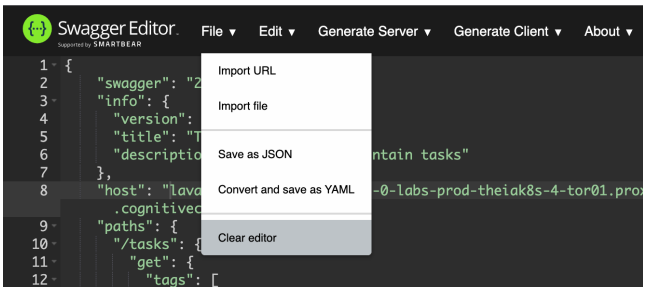
18. You can scroll down to view the output of the API call.



19. Try to do the following:

- Add a task
- Retrieve the tasks to see if your task is added to the list
- Get the details on one task
- Delete a task and check the list to verify that it is deleted.

20. From the **File** menu, click **Clear Editor** to clear the content of the Swagger Editor.



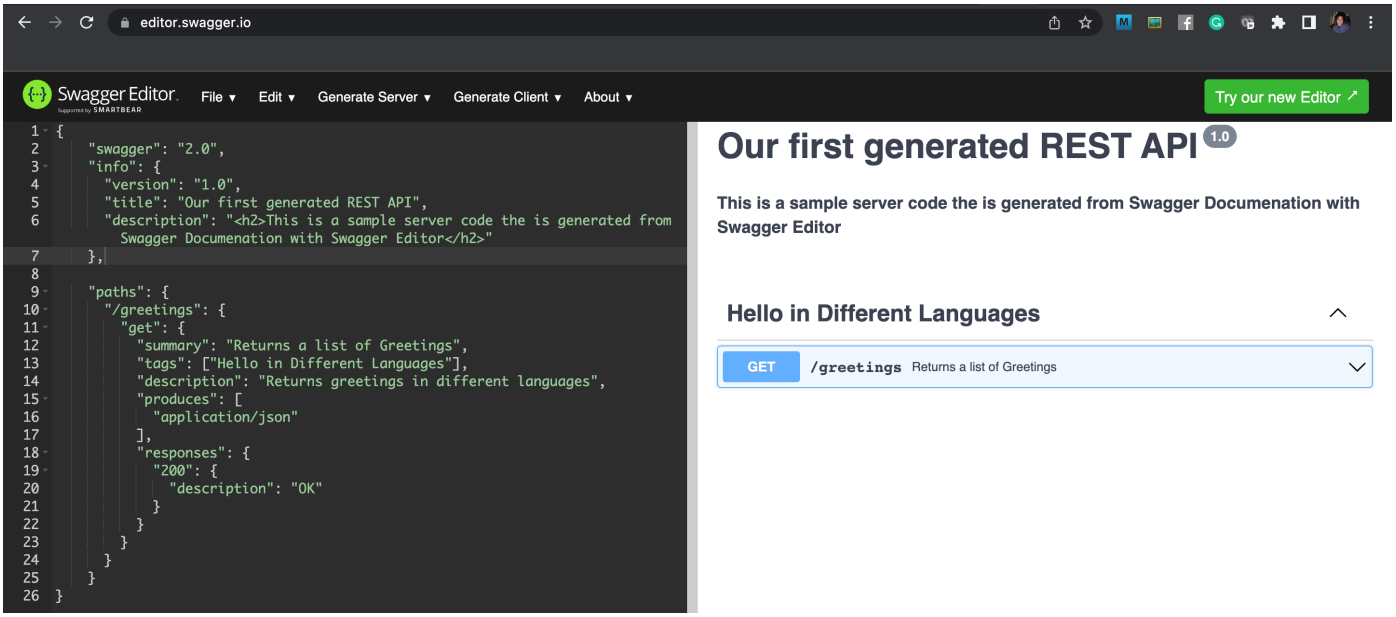
Task 2 - Creating Swagger documentation and generating server code

1. Now you will create a REST API with Swagger documentation. To start with, let's define your application.

- It will adhere to Swagger 2.0 version.
- This is the first version of the application.
- It will have one endpoint /greetings, which returns the list of greetings as a JSON object.

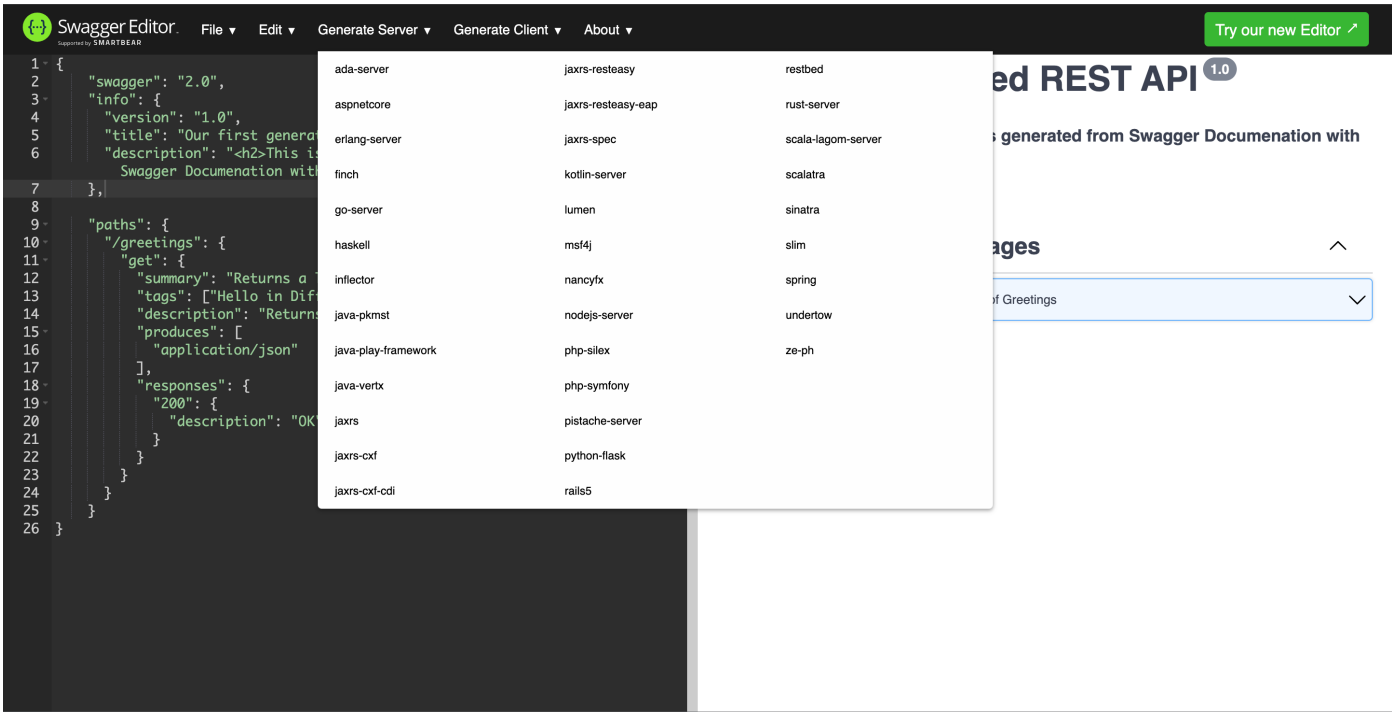
2. Copy and paste the following JSON in the Swagger Editor. You will get a prompt that says `Would you like to convert your JSON into YAML?`. Press `Cancel` to paste the content.

```
{
    "swagger": "2.0",
    "info": {
      "version": "1.0",
      "title": "Our first generated REST API",
      "description": "<h2>This is a sample server code the is generated from Swagger Documenation with Swagger Editor</h2>"
    },
    "paths": {
      "/greetings": {
        "get": {
          "summary": "Returns a list of Greetings",
          "tags": ["Hello in Different Languages"],
          "description": "Returns greetings in different languages",
          "produces": [
            "application/json"
          ],
          "responses": {
            "200": {
              "description": "OK"
            }
          }
        }
      }
    }
}
```



You will see the Swagger UI automatically appearing on the right. You cannot test it yet as your application is not defined and running yet.

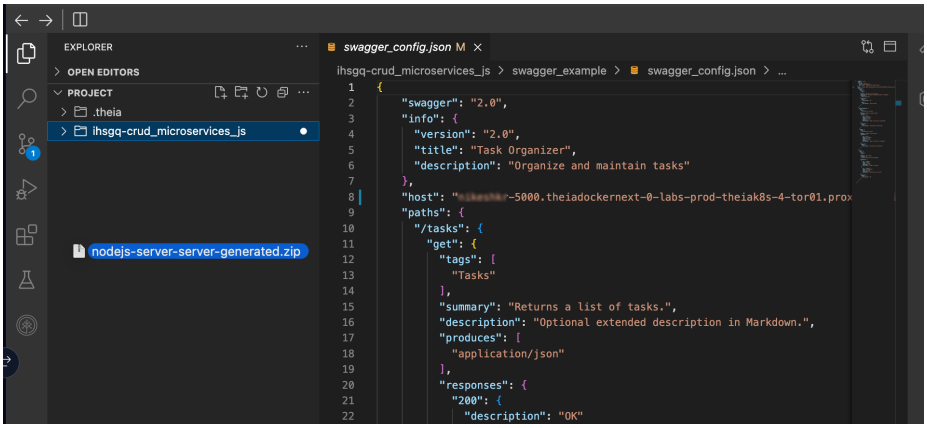
3. From the menu on top, click **Generate Server** and select **nodejs-server**. This will automatically generate the server code as a zip file named **nodejs-server-generated.zip**. Download the zip file to your system.



Note: If you are unable to download the ZIP file from Swagger or encounter errors while downloading, you can use the following command to download a working file directly:

```
wget -O nodejs-server-server-generated.zip "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/6YMK8TeeBFWY-MSGoQVLj-w/nodejs-server-server-generated.zip"
```

4. In your lab environment, click the **PROJECT** folder and drag and drop the zip file there.



5. On the terminal, go to the **/home/project** directory.

```
cd /home/project
```

6. Check to see if the zip file that you just dragged and dropped exists.

```
ls nodejs-server-server-generated.zip
```

7. Unzip the contents of the zip file into a directory named **nodejs-server-generated** by running the following command.

```
unzip nodejs-server-server-generated.zip -d nodejs-server-generated/
```

8. Change to the **nodejs-server** folder inside the folder you just extracted the zip file into.

```
cd nodejs-server-generated/nodejs-server-server
```

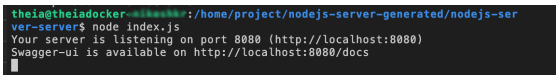
9. The entire server setup along with endpoint is done for you already. Let's install necessary dependancies.

```
npm install
```

10. Run the below commands to start node server. The server generated code automatically is configured to run on port 8080.

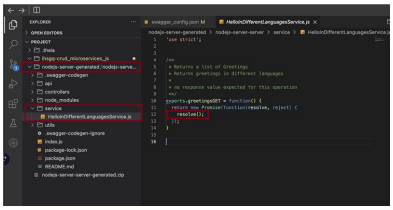
```
node index.js
```

This confirm that the node server is running.



11. Now you should stop the server by pressing `ctrl + C`.

12. In the file explorer, go to `nodejs-server-generated/nodejs-server-server/service/HelloInDifferentLanguagesService.js`. This is where you need to implement your actual response for the REST API.



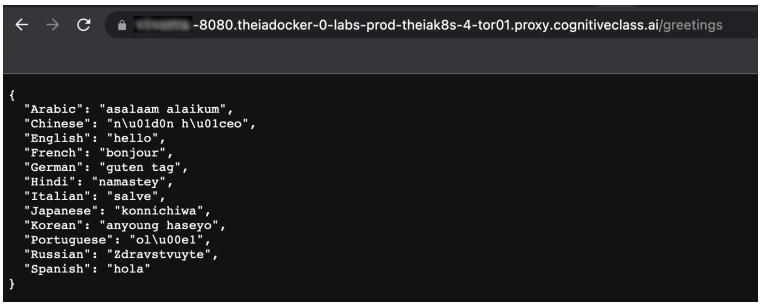
13. Replace `resolve();` with the following code.

```
let hellos = {
      "English": "hello",
      "Hindi": "namastay",
      "Spanish": "hola",
      "French": "bonjour",
      "German": "guten tag",
      "Italian": "salve",
      "Chinese": "nin hão",
      "Portuguese": "olá",
      "Arabic": "asalaam alaikum",
      "Japanese": "konnichiwa",
      "Korean": "anyoung haseyo",
      "Russian": "Zdravstvuyte"
    }
resolve(hellos);
```

14. Run the node server again.

```
node index.js
```

15. Click the Skills Network button on the left, it will open the "Skills Network Toolbox." Then click Launch Application, from there you enter the port no. as 8080 and click Your Application button. This will open a browser window. Append the path /**greetings** to the URL. You should see the greetings in the page.



Congratulations! You have successfully completed the task.

Author:

Rajashree Patil
Nikesh Kumar